PART 1

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
17	97%	203/210	65%	60/93	67%	60/89

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.yorku.booking	2	100%	44/44	25%	6/24	25%	6/24
com.yorku.command	3	100%	23/23	67%	2/3	67%	2/3
com.yorku.coordinator	2	94%	82/87	78%	32/41	86%	32/37
com.yorku.observer	2	100%	12/12	67%	2/3	67%	2/3
com.yorku.users	8	95%	42/44	82%	18/22	82%	18/22

Report generated by PIT 1.23.0

Enhanced functionality available at arcmutate.com

PART 2

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
17	80%	169/210	40%	37/93	58%	37/64

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.yorku.booking	2	64%	28/44	17%	4/24	29%	4/14
com.yorku.command	3	100%	23/23	67%	2/3	67%	2/3
com.yorku.coordinator	2	80%	70/87	34%	14/41	52%	14/27
com.yorku.observer	2	100%	12/12	67%	2/3	67%	2/3
com.yorku.users	8	82%	36/44	68%	15/22	88%	15/17

Report generated by PIT 1.23.0

Enhanced functionality available at arcmutate.com

PART 3
B.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
17	93%	196/210	70%	65/93	76%	65/85

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.yorku.booking	2	86%	38/44	38%	9/24	43%	9/21
com.yorku.command	3	100%	23/23	100%	3/3	100%	3/3
com.yorku.coordinator	2	97%	84/87	80%	33/41	85%	33/39
com.yorku.observer	2	100%	12/12	67%	2/3	67%	2/3
com.yorku.users	8	89%	39/44	82%	18/22	95%	18/19

Report generated by PIT 1.23.0

Enhanced functionality available at arcmutate.com

C. The AI assistant used was the github copilot. The AI responded to each prompt quickly but always seemed to added some unnecessary features to the project. Each test was accepted and the only modification was to add more tests for more coverage. No prompt refinement steps where needed. My understanding of the classes where at a level just above the AI. i believed that testing for as many situations as possible could be a better way to test the code but the AI insisted on the bare minimum. In the end half of the classes from each side were added. If the answer at run time was the same as my answer i deemed it correct. No test failed at run time or provided incorrect assertions and successful test cases that passed and provided meaningful, accurate coverage would be

1. extendReservationUpdatesReservedHours
2. testClonePreservesData

PART 4

	manually written test cases	Tests generated by the AI assistant	Tests generated by Randoop
Code coverage	Highest	Second highest	lowest
reason	Most likely best we intentionally target edge cases and	They systematically explored many paths But may miss very	Excels at brute-force exploration But coverage can

	known logic paths and some tests were deleted from randoop since they failed at run time	specific or domain-driven cases	include **irrelevant or redundant paths**
Mutation score	Second highest	highest	lowest
Readability	best	Second best	worst
reason	Clear structure,Easy to understand intent	Generally readable, but: Less semantic naming	Poor readability,Random method calls
Usefulness	highest	Second highest	low
reason	High practical value: Test real use cases	Good usefulness: Strong at catching edge cases	Low usefulness: Many tests are trivial or irrelevant